

Answer 1 a)**i)**

Well-formedness and Validity are two important concepts that defines the structure and correctness of the XML document. The contrast between both is shown below:

Well-formedness:

A well-formed XML document satisfies the basic syntax rules specified by the XML standard. It indicates that the document complies with the essential rules required for XML processors to read and parse the document.

Following are the seven rules that defines whether an XML document is well-formed or not -

- The document must only have one root element.
- Attribute values must be enclosed in quotes.
- Special characters like < or & must be escaped.
- All start tag must have a matching close tag.
- Nesting of elements must be proper and must not overlap.
- There must not be two attributes with same name for an element.
- No comments should appear inside the tags.

Validity:

A Valid XML document not only satisfies the above well-formedness rules but also adheres to a specific Document Type Definition (DTD) or XML Schema Definition (XSD). The relationships, data types, and structure of the XML document are specified by these extra rules.

A Valid XML document is always Well-formed, but a well-formed document is not necessarily valid. XML parsers can verify well-formedness, but validating parsers against a DTD or XML schema that specifies the permitted structure is the only way to verify validity.

ii)

In XML, the terms "document element" and "document root" refer to specific elements within an XML document, and they play distinct roles in defining the document's structure.

Document Element:

The document element is the outermost element in an XML document. It encloses all other elements and represents the root of the XML hierarchy. The content of the document element includes all other elements in the document. For example –

```
<bookstore>
  <book>
    <title>Sample Book</title>
    <author>John Doe</author>
  </book>
</bookstore>
```

Here the bookstore is the document element.

In XPath, the term document element refers to the element selected as the entry point for an XPath expression. It is often denoted by a forward slash (/) at the beginning of the XPath expression.

For example, the XPath expression “/bookstore/book” will select the document elements named “book” under “bookstore” in the XML document.

Document Root:

The document root is the top-level element that contains all other elements in an XML document. It is essentially the same as the document element.

The document root does not have a specific path reference in XPath. It is simply referenced by / as the parent of all other elements. For example, “/bookstore” expression will refer to the root element bookstore.

Answer 1 b)

From the given poppins.html output we can see we need to add additional xsl:template for processing the track elements in poppins.xml. We can add the following xsl in web.xsl after line 23.

```
<xsl:template match="track">
  <li>
    <b>
      <xsl:value-of select="name"/>
    </b>
    <xsl:text> by </xsl:text>
    <xsl:apply-templates select="by"/>
    <xsl:text> (</xsl:text>
    <xsl:value-of select="@min"/>
```

```

        <xsl:text>.</xsl:text>
        <xsl:value-of select="@sec"/>
        <xsl:text>)</xsl:text>
    </li>
</xsl:template>

<xsl:template match="by">
    <xsl:value-of select="."/>
    <xsl:if test="position() != last()">
        <xsl:text>,</xsl:text>
    </xsl:if>
</xsl:template>

```

The new <xsl:template> is added to handle the <track> elements and the nested <by> elements. The <by> template uses the position() and last() functions to check whether a comma should be added between multiple artists.

Answer 1 c)

To group track elements according to the number of contributing artists we can modify the existing stylesheet at line 17 by following code -

```

<xsl:template match="album">
    <h1>
        <xsl:value-of select="title"/>
    </h1>
    <xsl:for-each-group select="track" group-by="count(by)">
        <xsl:choose>
            <xsl:when test="current-grouping-key() = 1">
                <xsl:text>Tracks with 1 artist:</xsl:text>
            </xsl:when>
            <xsl:otherwise>
                <xsl:text>Tracks with </xsl:text>
                <xsl:value-of select="current-grouping-key()"/>
                <xsl:text> artists:</xsl:text>
            </xsl:otherwise>
        </xsl:choose>
    </xsl:for-each-group>
    <xsl:apply-templates select="current-group()"/>
</xsl:template>

```

Here, <xsl:for-each-group> is used to group the <track> elements based on the count of <by> elements within each track. The current-grouping-key() function is used to retrieve the count of artists for each group. The output will then display tracks grouped by the number of contributing artists.

Answer 2 a)**i)**

The three types of RDF Resource are as follows –

URI References:

A URI reference represents a resource that has a unique identifier in the form of a Uniform Resource Identifier (URI). It can be a web URL or any other URI that uniquely identifies the resource. URI references are written as complete URIs, for example, "<http://example.com/resource>"

Blank Node:

A blank node represents a resource for which no explicit identifier is provided. It is used when a resource doesn't have a URI but needs to be referenced within the RDF graph.

Literal:

A literal represents a data value associated with a property. It can be a string, number, date, or other types. Literals are used to provide values for properties associated with resources. Literals are enclosed in double quotes and may include a datatype or language tag.

ii)

The three ways of representing RDF metadata are as follows –

RDF Triplets:

RDF (Resource Description Framework) is based on the concept of triples, which consist of subject-predicate-object statements. Each triple represents a relationship between resources and is expressed as a subject, a predicate, and an object.

For example:

(subject) -> (predicate) -> (object)

(http://example.com/resource1) -> (dc:title) -> ("Sample Resource")

(http://example.com/resource1) -> (dc:creator) -> ("John Doe")

Graph-Based Representation:

RDF data can be visualized as a graph, where nodes represent resources, and edges represent the relationships between them. This graph-based representation provides a visual way to understand the interconnected nature of RDF data.

For example:

```
(http://example.com/resource1) --dc:title--> "Sample Resource"  
(http://example.com/resource1) --dc:creator--> "John Doe"
```

Serialization:

Serialization refers to the process of converting RDF data from graphs into a specific syntax or serial form that can be stored. Various serialization formats exist, such as RDF/XML, Turtle, JSON-LD, and N-Triples, each providing a different syntax for representing the same RDF information.

Answer 2 b)

i)

To list all pairs of artists that collaborated on a track together we can write a XQuery expression as follows –

```
for $track in /album/track  
let $artists := $track/by  
where count($artists) > 1  
return string-join($artists, ' and ')
```

This XQuery expression iterates over each <track> element and selects the <by> elements within the track. If there is more than one artist (count(\$artists) > 1), it returns a string concatenating the names of the collaborating artists using " and " as the separator.

ii)

Now to also count the number of tracks featuring each pair we can modify the expression as below –

```
let $tracks :=  
  for $track in /album/track  
  let $artists := $track/by  
  where count($artists) > 1  
  return string-join($artists, ' and ')
```

```

let $tracksCount :=
  for $pair in distinct-values($tracks)
  return
    <pair>
      <artists>{ $pair }</artists>
      <count>{ count($tracks[. eq $pair]) }</count>
    </pair>

return $tracksCount

```

Here, the *let \$tracks* clause generates a sequence of artist pairs based on the original query. The *let \$tracksCount* clause counts the number of tracks featuring each unique pair of artists using *distinct-values* and *count*.

Answer 2 c)

To represent the metadata in an RDF graph with a single blank node we can write as follows -

```

@prefix rdf: <http://www.w3.org/>.
@prefix ex: <http://example.com/>.

ex:blankNode rdf:type rdf:BlankNode;
  ex:hasName "Prince Rodgers Nelson";
  ex:wasBornIn ex:Minneapolis;
  ex:wasBornOnYear 1958;
  ex:isArtistOf ex:1999.

ex:1999 rdf:type ex:Song;
  ex:wasWrittenBy ex:Prince;
  ex:wasProducedBy ex:Prince;
  ex:wasPerformedBy ex:Prince.

ex:Prince rdf:type ex:Person;
  ex:hasName "Prince".

```

Here, *ex:blankNode* is a blank node representing information about Prince, the song "1999," and his birthplace. The song "1999" is connected to Prince through properties like *ex:wasWrittenBy*, *ex:wasProducedBy*, and *ex:wasPerformedBy*. *ex:Prince* is another node representing Prince with his name.

Answer 3 a)**Replication:**

In a distributed system, replication is the process of making and keeping duplicate copies of data on several nodes or servers. Each replica contains the same information, and updates to one copy are propagated to the others.

Following are some advantages of using replication:

- **High Availability** - By creating redundant copies, replication raises the availability of the system. Continuous service can be ensured by having a backup replica in case of node failure.
- **Fault Tolerance** - Having multiple copies of the data guarantees that the system can continue to operate without losing any data in the case of a hardware malfunction or network problem.
- **Read Scalability** - By dividing up read queries among several replicas, replication enables load balancing. Because each replica can handle a portion of the read traffic, read performance is improved.

Sharding:

Sharding involves partitioning a database into smaller, independent units called shards. A portion of the data falls under the purview of each shard. Data is divided using sharding, a horizontal partitioning technique, according to predetermined standards.

Following are some advantages of using sharding:

- **Better Write Scalability** - Sharding makes writing more scalable by dividing up write operations among several shards. There will be less contention for a single write source since each shard can handle its share of the write load.
- **Storage Efficiency** - Sharding makes it possible to use storage resources more effectively. Data is divided among smaller servers, or clusters, which can be more affordable and manageable, as opposed to a single, massive server.
- **Enhanced Throughput** - Sharding can enhance system throughput by dividing workload and data among several shards. Because each shard runs independently, queries can be processed in parallel.

Answer 3 b)

A replica set in a NoSQL database like MongoDB is a group of servers that maintain identical copies of data. It provides high availability and fault tolerance by allowing automatic recovery in the event of a primary node failure. The replica set architecture ensures data consistency and enables read scaling by distributing read queries among the secondary nodes.

Following are components of a MongoDB replica set:

- **Primary Node** - The primary node handles all write operations and applies the changes to its data set. It replicates these changes to the secondary nodes asynchronously.
- **Secondary Node** - These nodes are replicas of the primary node that maintain copies of the data and apply the operations to their data set. They can serve read queries but not writes.
- **Oplog (Operation Log)** - The oplog is a special collection that records all write operations in the order they occur on the primary. Secondary nodes use the oplog to replicate these operations and stay synchronized with the primary.
- **Arbiter** - This node is a special node that participates in the election process but does not store a full copy of the data. It helps in achieving a majority vote without consuming significant resources.
- **Heartbeat and Election** - Nodes in a MongoDB replica set communicate with each other through heartbeat messages. If a node does not receive a heartbeat within a specified time frame, it may trigger an election process to choose a new primary node from the available secondaries.
- **Replication Process** - MongoDB uses asynchronous replication to keep secondary nodes up-to-date with the primary. The replication process involves the primary sending the oplog entries to the secondaries, which then apply those operations to their own data.

Answer 3 c)**i) Acquiring a NoSQL DBMS:**

Strengths:

- NoSQL databases, such as MongoDB, offer schema flexibility, making it easier to handle complex and dynamic data structures, like those found in sales orders with multiple components.
- NoSQL databases excel at horizontal scalability, allowing the startup to handle growing data and user loads efficiently, crucial for a web-based sales operation.
- These databases are designed to handle large amounts of unstructured or semi-structured data efficiently, providing good performance for read and write operations.

Weaknesses:

- Some NoSQL databases prioritize eventual consistency, which might be a drawback for applications where immediate consistency is critical.
- NoSQL databases typically don't support SQL queries, which might be a challenge when interfacing with suppliers using relational systems.

ii) Acquiring a Relational DBMS:

Strengths:

- Relational databases provide ACID properties (Atomicity, Consistency, Isolation, Durability), ensuring data integrity, which is crucial for sales order documentation.
- SQL is a widely-used and standardized query language, making it easier for the organization to find skilled developers and integrate with suppliers using relational systems.

Weaknesses:

- Relational databases have a fixed schema, which can be a limitation when dealing with complex and evolving data structures.
- Vertical scalability is a common limitation for relational databases, which might pose challenges as the organization grows.

- Relational databases are not as well-suited for handling unstructured or semi-structured data, which may be present in sales orders with multiple components.

iii) Going for a Hybrid Solution:

Strengths:

- A hybrid solution allows the organization to leverage the strengths of both relational and NoSQL databases. They can use a relational database for structured data and a NoSQL database for handling unstructured or semi-structured data.
- It provides flexibility to choose the appropriate database model based on the nature of the data, offering a tailored approach to different data requirements.

Weaknesses:

- Managing a hybrid solution introduces complexity in terms of integration, maintenance, and potentially increased costs.
- The organization needs expertise in both relational and NoSQL databases, which might be a challenge in terms of hiring or training.

Final Recommendation:

Considering the strengths and weaknesses of both databases, my recommendation would be to opt for hybrid solution. With this strategy, the company can effectively handle the variety of data requirements by utilising the advantages of both database models. Whereas a NoSQL database can effectively handle the complexity of sales orders with multiple components, relational databases can handle structured data and satisfy customs requirements.

Answer 4 a)

MongoDB provides the following mechanisms for modelling complex documents and relationships –

Embedded Documents: MongoDB allows embedding documents within documents, providing a way to model complex structures within a single document. For example –

```
{
  _id: "1234",
  name: "Jay Agrawal",
  address: {
    street: "123 Main St",
    city: "London",
  }
}
```

Here, "address" field contain embedded document representing address details.

Document References: MongoDB supports referencing or linking documents using references to establish relationships between them. For example –

```
{
  "_id": 1,
  "name": "Jay",
  "addressId": ObjectId("address123")
}

{
  "_id": ObjectId("address123"),
  "street": "456 Oak Street",
  "city": "London",
  "country": "UK"
}
```

Here, the "addressId" field in the User document references the ObjectId of the related Address document.

Arrays and Lists: MongoDB also allows the use of arrays to model lists of values or documents, providing flexibility for handling multiple items within a single field. For example –

```
{
  "_id": 1,
  "name": "Jay Agrawal",
  "interests": ["Reading", "Traveling", "Photography"]
}
```

Here, the "interests" field is an array containing a list of interests associated with the user.

Dynamic Schema: MongoDB has a dynamic schema, allowing documents in the same collection to have different fields. This flexibility is useful for modeling varying data. For example –

```
{
  "_id": 1,
  "name": "xyz",
  "age": 30,
  "city": "New York"
}

{
  "_id": 2,
  "name": "abc",
  "hobbies": ["Swimming", "Gaming"],
  "city": "London"
}
```

Here, 1st document has "age" and 2nd document has "hobbies," demonstrating the dynamic nature of the schema.

Answer 3 b)

To investigate and address the slow queries in the product-orders database in MongoDB, we can use several tools and techniques. Following are some tools provided by MongoDB that we can utilize:

- MongoDB Profiler - It allows us to capture data about the operations happening within the database. This would show the slow running queries and execution stats like documents returned, indexes used etc.
- Explain Method - The explain() method can be used on slow queries to get query execution details like winning plan, indexes used, document scans required, etc. This would reveal why certain queries are slow.
- Index Usage - The index usage statistics can show me which indexes are being hit by queries and which are unused. This helps determine if the right indexes exist and are being leveraged.
- Schema Analysis - Using tools like the MongoDB Compass, I can visually analyze the schema, examine relationships, observe document growth over time. This can reveal complex nested structures that are contributing to slow queries.

- Client-side Tracing - Integrations like MongoDB .NET driver allow tracing queries from the application code to get granular performance data on the client-side.

By using these tools I would be able to point out queries that are slow, understand why, and use that to optimize schema, add indexes, or tune the queries as needed.

Answer 3 c)

Here are two strategies I would use to resolve the performance issues caused by a large orders collection with complex product documents:

1. Data Sharding

Sharding refers to horizontally partitioning the orders collection data across multiple MongoDB servers. This would split the giant orders collection into smaller chunks spread across shards. It allows storing larger data volumes, while queries hit only the shards containing relevant data, spreading load.

Indexing and queries would also be faster as indexes are smaller on each shard. Adding more shards scales system capacity easily.

2. Normalize the Product Embeddings

The order documents likely have large product sub-documents nested inside them. By normalizing some of these embeddings into a separate Products collection, the order documents can use leaner document references instead.

Smaller Order documents would improve query, update and scan speeds. The Products collection can be sharded independently as needed. Joins between orders and products can retrieve data in separate queries fairly efficiently.

This balances normalization with some embedding for performance. Queries avoid pulling overly large documents. Both sharding and normalization would help resolve the issues faced due to large, complex document growth in a single Orders collection. Sharding helps scale horizontally while normalization helps reduce and separate data to simplify querying

Answer 3 d)

Following are the two advantages of MongoDB being schemaless –

- Flexibility - Documents can have varied and dynamic structures in the same collection without needing to enforce a strict schema. This is useful for semi-structured data. New document types and fields can easily be added as needed without any schema migrations.
- Agility - Developers can make changes to the structure of data models on the fly to meet changing app needs without the overhead of managing schema migrations. This leads to faster iterations.

The one Drawback is data Inconsistency. The flexibility of schemaless and dynamic documents means there is possibility for data inconsistency. For example, some documents may have missing required fields that other documents contain or have incompatible data types in the same field. Enforcing regulatory rules also becomes more difficult.